

LexiScan Auto

AI Contract Intelligence Platform

Weekly Project Progress Report

Zaalima Internship · Generated 09 June 2026

This report documents the weekly progress on **LexiScan Auto**, a full-stack AI application that reads legal contract PDFs and extracts structured intelligence — named entities, a machine-learning–predicted contract type, confidence and risk scores, and a PII-redacted copy of the document — behind a secure, authenticated dashboard.

Project Overview

Technology Stack

Backend	FastAPI · Uvicorn · SQLite · Pydantic
AI / NLP	Spacy NER · PyMuPDF · scikit-learn (TF-IDF + LogisticRegression)
Auth	PyJWT (JWT) · bcrypt password hashing
Frontend	React 19 · Vite · custom glassmorphism CSS
Reporting	reportlab (this PDF)

Key Features Delivered

NER Extraction	Parties, dates, amounts, locations, termination clauses
ML Type Prediction	Offline classifier across 6 contract categories
PII Redaction	Automatic masking of names & financial figures
Confidence & Risk	0–100 score with LOW/MEDIUM/HIGH and review flags
Authentication	JWT + bcrypt; per-user private history
Analytics	Risk distribution, type breakdown, processing metrics

Weekly Breakdown

Week 1 — Foundation & AI Pipeline

Goal: Stand up the backend and the core NLP extraction engine.

- Initialised the project structure (ai / backend / frontend) and Git repository.
- Built the FastAPI backend with CORS and a SQLite persistence layer.
- Implemented the PDF text-extraction pipeline using PyMuPDF (no OCR).
- Integrated Spacy Named Entity Recognition to extract parties, dates, amounts and locations.
- Added rule-based termination-clause detection with regex patterns.
- Created the POST /extract endpoint and a sample contract for testing.

Deliverable: Working API that turns a PDF into structured entities.

Week 2 — Dashboard, Scoring & PII Redaction

Goal: Build the React dashboard and add intelligence on top of raw entities.

- Built the React + Vite single-page dashboard with a glassmorphism design system.
- Implemented drag-and-drop PDF upload with an animated processing pipeline.
- Added a 0–100 AI confidence score and LOW / MEDIUM / HIGH risk assessment.
- Built the PII-redaction engine that masks party names and financial figures.
- Added multi-tab navigation: Upload, Results, History and Stats.
- Implemented full CRUD (list / detail / delete) and an analytics dashboard.

Deliverable: End-to-end app: upload a contract, see scored, redacted results.

Week 3 — ML Prediction, Authentication & Landing Page

Goal: Add a machine-learning model, secure the app, and ship a public landing page.

- Built an offline contract-type classifier (TF-IDF + Logistic Regression, scikit-learn) predicting NDA / Employment / Lease / Service / Sales / Loan — no external API.
- Wired the prediction into the pipeline, results view, history table and stats.
- Added JWT authentication (PyJWT) with bcrypt-hashed passwords and a users table.
- Scoped all contracts to the authenticated user; protected every data endpoint.
- Built register / login screens and token-based session handling on the frontend.
- Designed and built a marketing landing page (hero, features, how-it-works, CTA).
- Added this automated weekly-report PDF generator (reportlab).

Deliverable: Secure, multi-user product with ML prediction and a public landing page.

Outcome

Over three weeks the project grew from an empty repository into a secure, multi-user AI product. It combines classical NLP (Spacy NER, PyMuPDF) with a custom machine-learning classifier for contract-type prediction, all running fully on-device with no external AI API. The application is protected by JWT authentication, scopes data per user, and ships with a public landing page and automated PDF reporting.